

15 Join Strategies

When you write a join in Spark, Spark decides **how** to execute it.

This decision is based on:

- data size
- join keys
- memory availability
- AQE runtime statistics

What does “Hash” mean in Spark joins?

Hash = building an in memory hash table on the join key.

- Join key is passed through a hash function
- Hash value decides bucket location
- Lookup becomes constant time (O1)

Spark uses hashing **only when it is safe and efficient.**

1. Broadcast Hash Join

When used

- One side is small enough to fit in executor as well as driver memory

How it works

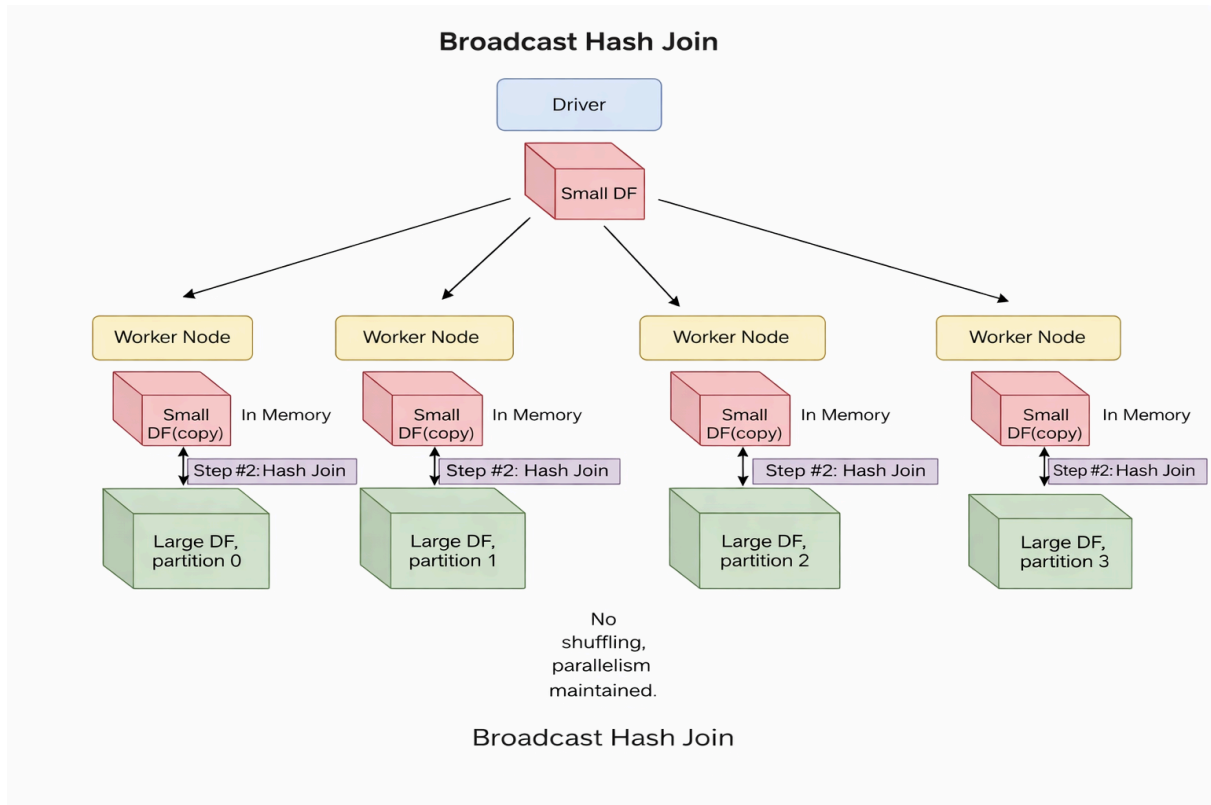
- Small table → converted into hash table
- Hash table → broadcasted to all executors
- Large table → scanned and probed against hash table

Why hash helps

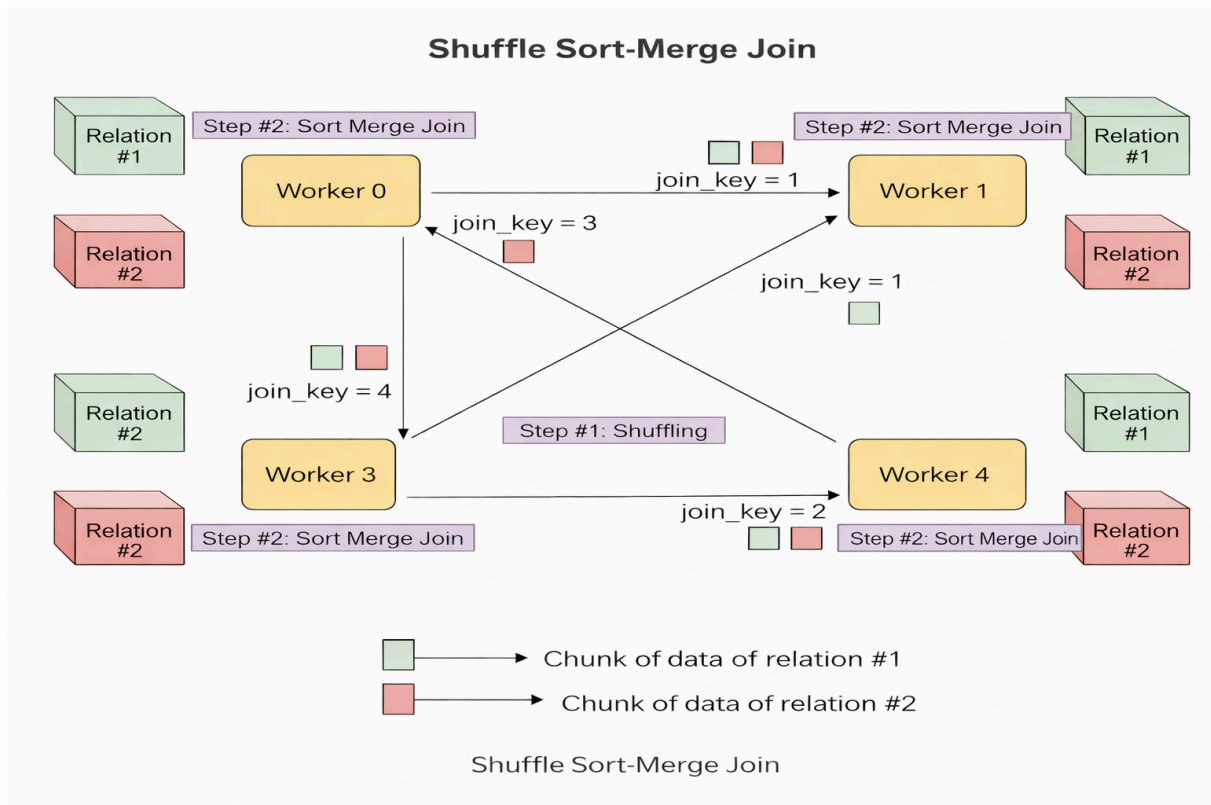
- Instant lookup
- No shuffle
- Fastest join strategy

Risk

- If hash table is too big → OOM



2. Shuffle Sort Merge Join



When used

- Both tables are large
- Join keys are sortable

How it works

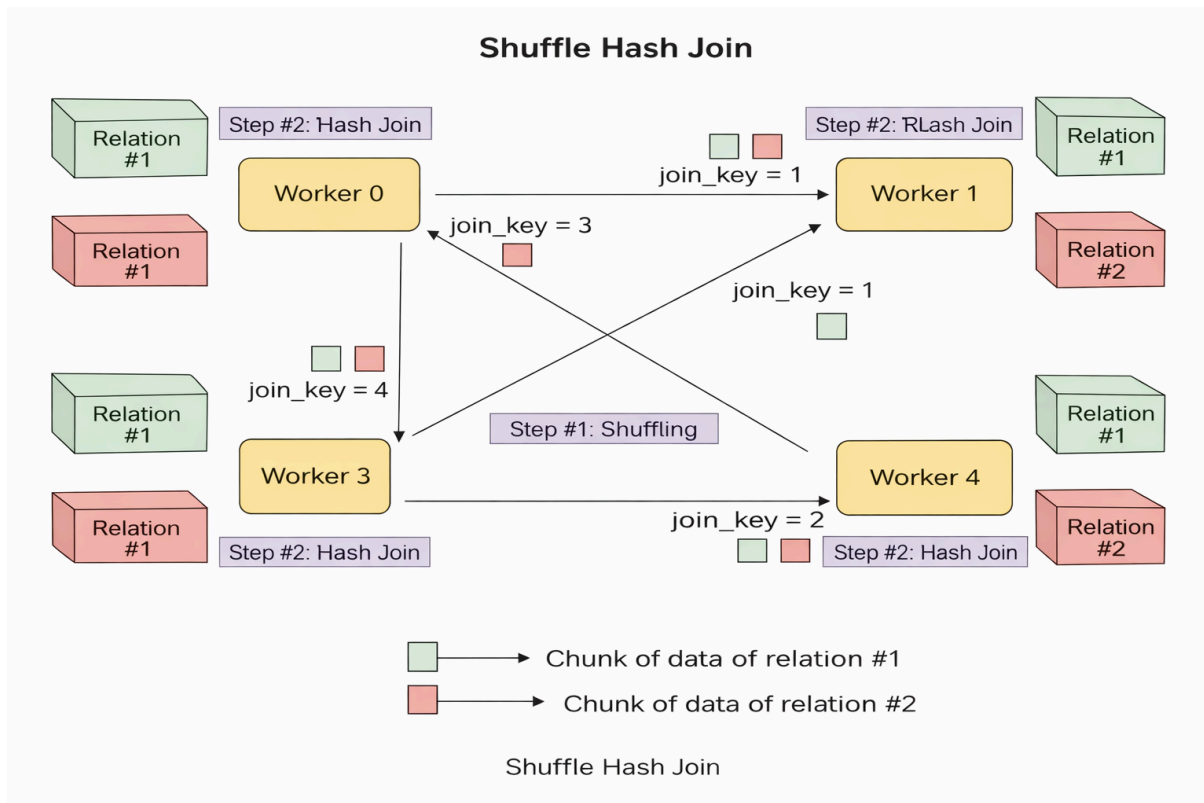
- Shuffle both tables
- Sort by join key
- Merge matching rows sequentially

Why no hash here

- Hash tables need memory
- Sorting allows streaming
- Safer for very large datasets

Default strategy for big data joins.

3. Shuffle Hash Join



When used

- 1 tables is large and another table is medium sized
- One side is smaller but cannot be broadcasted

How it works

- Both tables are shuffled by join key
- Smaller side builds hash table per partition
- Larger side probes that hash table

Hash usage

- Hash table is **local to executor**
- Not broadcasted

⚠ Less common in modern Spark, often replaced by Sort Merge Join.

```
from pyspark.sql.types import *
from pyspark.sql.functions import *
orders_schema_struct = StructType(
[
    StructField("order_id",IntegerType(),False)
```

```

,StructField("customer_id",IntegerType(),False)
,StructField("product_id",IntegerType(),False)
,StructField("unit_price",FloatType(),False)
,StructField("order_date",DateType(),False)
,StructField("order_status",StringType(),False)
,StructField("state",StringType(),False)
,StructField("quantity",IntegerType(),False)
]
)

df_orders = spark.read.csv("/data/orders_1gb.csv" , header = True , schem
a = orders_schema_struct)

product_schema = 'product_id int , cost_price float'

df_products = spark.read.csv("/data/products_12mb.csv" , header = True ,
schema = product_schema)

df_orders.show()
df_products.show()
# shuffle sort merge
df_orders.join(df_products, "product_id", "inner").write.mode("overwrite").f
ormat("noop").save()

# broadcast
df_orders.join(broadcast(df_products), "product_id", "inner").write.mode("o
verwrite").format("noop").save()

# shuffle hash
df_orders.join(df_products.hint("shuffle_hash"), "product_id", "inner").writ
e.mode("overwrite").format("noop").save()

```

Bucket Join : Joining 2 large tables

Bucketing is used primarily to improve the performance of join operations.

Before you perform a join, you'll bucket your records on the join column. Spark is then able to figure out the right bucket where your join records live. While performing the join, Spark does not need to scan the entire data set to find the correct record.

You should create bucket on same column and same number of buckets for both the datasets

```
# bucketing can be done with table only
# works with parquet format only

df_orders.write \
.mode("overwrite") \
.bucketBy(8,"product_id") \
.sortBy("product_id") \
.saveAsTable("orders_bucket")

df_products.write \
.mode("overwrite") \
.bucketBy(8,"product_id") \
.sortBy("product_id") \
.saveAsTable("products_bucket")

#spark.sql("select * from orders_bucket o inner join products_bucket p on
o.product_id=p.product_id").write.format("noop").mode("overwrite").save()

df_orders_bucket = spark.table("orders_bucket")
df_products_bucket = spark.table("products_bucket")

df_orders_bucket.join(df_products_bucket, "product_id", "inner").write.mod
e("overwrite").format("noop").save()
```